NHẬP MÔN MẠNG NƠ-RON NHÂN TẠO

PERCEPTRON

Viên gạch đầu tiên của Mạng Nơ-ron Nhân tạo

Deep Learning từ A đến Z

Nội Dung Chính

1. Perceptron là gì, và nó ra đời để giải quyết vấn đề gì?

2. Vì sao Perceptron được gọi là "Artificial Neuron"?

3. Cấu tạo của Perceptron: Input → Weight → Activation → Output

4. Lan truyền xuôi (Forward Propagation) qua Perceptron

5. Ví dụ dễ hiểu cách tính toán

6. Cách Perceptron học

MỤC LỤC

PHẦN 1: KHÁI NIỆM PERCEPTRON

1 Perceptron Là Gì?

- 1.1 Tại sao cần Perceptron?
- 1.2 Định nghĩa Perceptron
- 1.3 Ví dụ
- 1.4 Tại sao Perceptron được gọi là "Artificial Neuron"?

PHẦN 2: BẢN CHẤT PERCEPTRON

2 Cấu Tạo Perceptron Và Lan Truyền Xuôi

- 2.1 Sơ đồ cấu tạo tổng quát
- 2.2 Giải thích chi tiết từng thành phần
  - 2.2.1 Input (Đầu vào)
  - 2.2.2 Weighted Sum (Tổ hợp tuyến tính)
  - 2.2.3 Activation (Hàm kích hoạt)
  - 2.2.4 Output (Đầu ra)
- 2.3 Tóm tắt quy trình lan truyền xuôi
- 2.4 Perceptron giải quyết vấn đề gì?

3 Magic Hàm Kích Hoạt

- 3.1 Bỏ hàm kích hoạt, ta được Linear Regression
- 3.2 Thay Step bằng Sigmoid, ta được Logistic Regression
- 3.3 Kết luận

4 Perceptron Học Như Thế Nào?

- 4.1 Ý tưởng cốt lõi
- 4.2 Công thức cập nhật trọng số
- 4.3 Các bước của thuật toán

5 Tổng Kết Bài Học

PHẦN 1: KHÁI NIỆM PERCEPTRON

1 Perceptron Là Gì?

Mọi công nghệ đều được tạo ra để giải quyết một vấn đề nào đó. Perceptron cũng không ngoại lệ. Trước khi đi vào định nghĩa, hãy cùng xem nó ra đời để giải quyết vấn đề gì.

1.1 Tại sao cần Perceptron?

Máy tính làm thế nào để tự đưa ra quyết định? Nghe có vẻ đơn giản, nhưng đây từng là một bài toán rất lớn trong lịch sử của ngành Trí tuệ nhân tạo.

Giả sử bạn muốn xây dựng một chương trình để xét duyệt hồ sơ vay vốn của ngân hàng. Với một lập trình viên, cách nghĩ đầu tiên là viết luật: **Cho vay hay không cho vay?** Nếu chỉ có một vài quy tắc đơn giản, ta hoàn toàn có thể lập trình theo cách thông thường:

```
nếu thu nhập > 30 triệu
và nợ hiện tại < 20 triệu
→ Cho vay

Ngược lại:
→ Từ chối
```

Cách làm này hoạt động tốt khi số lượng quy tắc còn ít. Tuy nhiên, trong thực tế, một khách hàng có thể được đánh giá dựa trên hàng chục, thậm chí hàng trăm yếu tố khác nhau như: tuổi, thu nhập, lịch sử tín dụng, nghề nghiệp, tài sản thế chấp, số người phụ thuộc…

Khi đó, việc tự viết hàng nghìn quy tắc *if…else…* gần như không còn khả thi. Không chỉ tốn nhiều thời gian và công sức, các quy tắc này còn rất khó bảo trì, đồng thời không thể tự thích nghi khi dữ liệu hoặc bối cảnh thay đổi.

Quan trọng hơn, vấn đề không chỉ nằm ở **số lượng quy tắc cần viết**, mà ở chỗ con người thường **không biết trước tất cả các quy tắc cần thiết** để giải quyết một bài toán. Nhiều mối quan hệ trong dữ liệu quá phức tạp để có thể diễn đạt đầy đủ bằng các luật lập trình.

Điều đó dẫn đến một ý tưởng hoàn toàn mới: thay vì lập trình sẵn mọi quy tắc, liệu có thể xây dựng một mô hình có khả năng **tự học các quy tắc từ dữ liệu** hay không?

Perceptron chính là một trong những mô hình đầu tiên hiện thực hóa ý tưởng này.

Mặc dù còn khá đơn giản so với các mô hình AI hiện đại, Perceptron đã đặt nền móng cho sự ra đời của mạng nơ-ron nhân tạo (*Artificial Neural Network*) và sau này là *Deep Learning*.

1.2 Định nghĩa Perceptron

Perceptron là một trong những thuật toán học máy có giám sát (*supervised learning*) đơn giản nhất, đồng thời là **đơn vị tính toán cơ bản** của mạng nơ-ron nhân tạo.

Về bản chất, Perceptron nhận vào một hoặc nhiều giá trị số (đầu vào), nhân mỗi giá trị với một **trọng số (weight)** thể hiện mức độ quan trọng của nó, cộng tất cả lại, rồi đưa qua một "bộ quyết định" để cho ra kết quả cuối cùng — thường là 0 hoặc 1.

> **Hiểu đơn giản:**
>
> *Có thể hình dung Perceptron như một "cỗ máy bỏ phiếu có trọng số": mỗi đầu vào là một lá phiếu, mỗi trọng số là mức độ ảnh hưởng của lá phiếu đó, và kết quả cuối cùng phụ thuộc vào tổng số phiếu có vượt qua một ngưỡng nhất định hay không.*

1.3 Ví dụ

Giả sử ngân hàng đã có dữ liệu lịch sử của những khách hàng trước đây:

Khách hàng	Tuổi	Thu nhập (triệu/tháng)	Nợ hiện tại (triệu)	Kết quả
A	28	45	10	✔ Cho vay
B	24	18	35	✘ Từ chối
C	35	60	5	✔ Cho vay
D	42	25	40	✘ Từ chối
E	31	50	15	✔ Cho vay
F	27	20	30	✘ Từ chối
…	…	…	…	…

Bảng 1.1: Dữ liệu lịch sử khách hàng (minh họa)

Dựa trên bộ dữ liệu lịch sử này, trong quá trình huấn luyện, Perceptron sẽ tự học mối quan hệ giữa các đặc trưng và kết quả quyết định. Nói cách khác, mô hình sẽ dần tìm câu trả lời cho các câu hỏi:

- Thu nhập có quan trọng hơn tuổi không?
- Khoản nợ ảnh hưởng đến quyết định nhiều hay ít?
- Khi kết hợp tất cả các yếu tố lại thì nên cho vay hay từ chối?

Sau quá trình huấn luyện, Perceptron sẽ biểu diễn mức độ ảnh hưởng của từng đặc trưng thông qua các trọng số (weights). Ví dụ, mô hình có thể học được rằng thu nhập có ảnh hưởng tích cực đến khả năng được duyệt vay, trong khi khoản nợ hiện tại làm giảm khả năng đó.

Điều cần nhấn mạnh là **Perceptron không học thuộc từng khách hàng**, mà học **quy luật chung** từ toàn bộ dữ liệu lịch sử để có thể đưa ra dự đoán cho những khách hàng chưa từng xuất hiện trước đó.

Đó cũng chính là ý nghĩa của chữ **"Learning"** trong **Machine Learning** và **Deep Learning**:

Học Từ Dữ Liệu.

1.4 Tại sao Perceptron được gọi là "Artificial Neuron"?

Cái tên "Artificial Neuron" (nơ-ron nhân tạo) xuất phát từ việc cấu trúc và cách hoạt động của Perceptron được xây dựng dựa trên ý tưởng mô phỏng lại tế bào thần kinh sinh học trong não bộ con người. Bảng so sánh dưới đây cho thấy sự tương đồng giữa hai đối tượng này:

Đặc điểm	Neuron sinh học	Perceptron (Artificial Neuron)
Tín hiệu vào	Xung điện từ các nhánh dendrite (nhánh nhận tín hiệu)	Các giá trị số x_1, x_2, ..., x_n
Mức độ ảnh hưởng	Độ nhạy của từng khớp thần kinh (synapse)	Trọng số w_1, w_2, ..., w_n
Ngưỡng kích hoạt	Neuron chỉ "phát xung" khi tổng tín hiệu vượt ngưỡng	Hàm kích hoạt (activation) áp dụng lên tổng có trọng số
Tín hiệu ra	Xung thần kinh truyền qua axon đến neuron khác	Giá trị output ŷ truyền tới lớp tiếp theo

Bảng 1.2: So sánh Neuron sinh học và Perceptron

Một neuron sinh học nhận tín hiệu từ nhiều neuron khác, mỗi tín hiệu có độ mạnh yếu khác nhau tùy theo khớp thần kinh. Khi tổng tín hiệu đủ lớn, neuron sẽ "phát xung" truyền tiếp đi. Perceptron làm điều tương tự bằng toán học: "*tổng hợp tín hiệu rồi ra quyết định*". Chính vì vậy, Perceptron được gọi là "Nơ-ron nhân tạo".

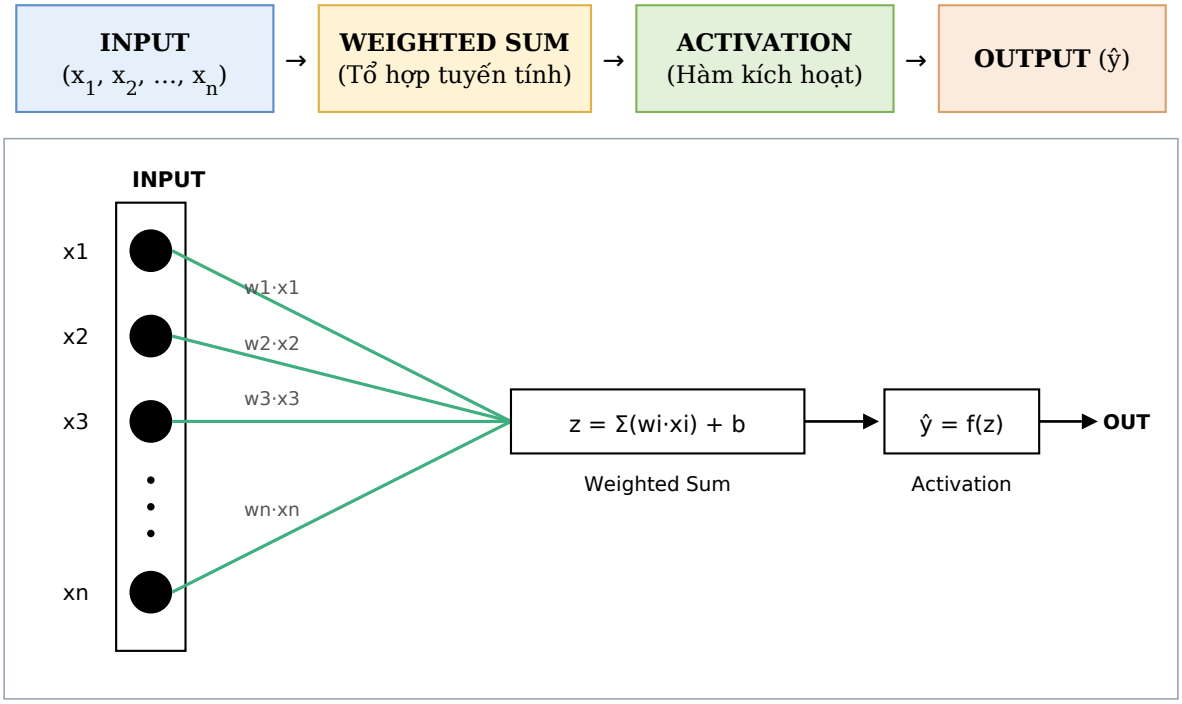
PHẦN 2: BẢN CHẤT PERCEPTRON

2 Cấu Tạo Perceptron Và Lan Truyền Xuôi

Sau khi đã hiểu Perceptron là gì và vai trò của nó, phần này sẽ đi sâu vào cấu tạo bên trong và cách một Perceptron xử lý thông tin để biến dữ liệu đầu vào thành kết quả đầu ra — quá trình này được gọi là lan truyền xuôi (forward propagation).

2.1 Sơ đồ cấu tạo tổng quát

Một Perceptron được cấu tạo từ bốn thành phần chính, xử lý dữ liệu theo một chiều duy nhất như sơ đồ dưới đây:



Hình 2.1: Quá trình lan truyền xuôi của Perceptron

Quá trình này được gọi là **Lan truyền xuôi (Forward Propagation)**, vì thông tin chỉ di chuyển theo một chiều duy nhất: từ lớp đầu vào (Input), qua các phép tính trong mô hình và đến lớp đầu ra (Output), không quay ngược trở lại.

2.2 Giải thích chi tiết từng thành phần

Để hiểu rõ từng thành phần, ta sẽ phân tích qua ví dụ: **Dự đoán khách hàng có mua xe hay không**. Giả sử một đại lý ô tô muốn dự đoán một khách hàng có mua xe hay không dựa trên một số thông tin cơ bản. Dữ liệu lịch sử của các khách hàng trước được thu thập như sau:

Khách hàng	Tuổi	Thu nhập (triệu/tháng)	Đã kết hôn	Có ô tô cũ	Mua xe (Label)
A	24	8	Không	Không	Không
B	29	15	Có	Không	Có
C	35	22	Có	Có	Có

D	27	10	Không	Không	Không
E	41	30	Có	Có	Có
F	33	18	Có	Không	Có

Bảng 2.1: Bộ dữ liệu khách hàng mua xe

Trong đó:

- **Tuổi, Thu nhập, Đã kết hôn** và **Có ô tô cũ** là các **đặc trưng (Features)**.
- **Mua xe** là **nhãn (Label)** mà mô hình cần dự đoán.
- Ta quy ước: **Có = 1**, **Không = 0**.

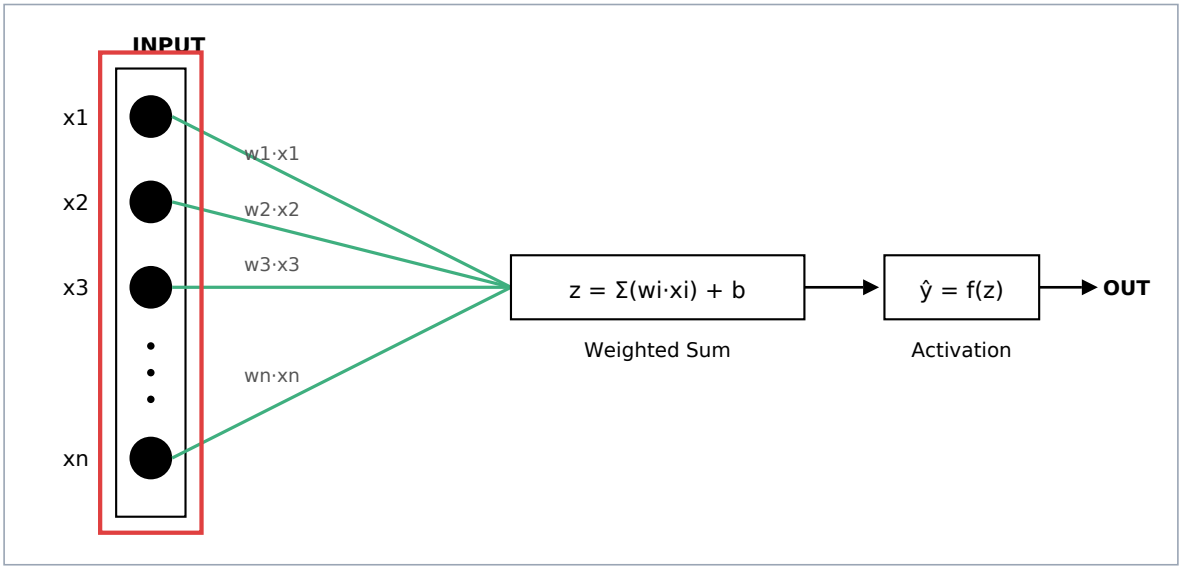
Sau khi mã hóa dữ liệu, bảng sẽ trở thành:

Khách hàng	x_1 (Tuổi)	x_2 (Thu nhập)	x_3 (Kết hôn)	x_4 (Ô tô cũ)	y
A	24	8	0	0	0
B	29	15	1	0	1
C	35	22	1	1	1
D	27	10	0	0	0
E	41	30	1	1	1
F	33	18	1	0	1

Bảng 2.2: Bộ dữ liệu sau khi mã hóa

2.2.1 Input (Đầu vào)

Mỗi hàng trong Bảng 2.2 đại diện cho một khách hàng, còn mỗi x_1, x_2, x_3, x_4 là một đặc trưng mô tả khách hàng đó. Khi đưa dữ liệu vào Perceptron, các đặc trưng này sẽ trở thành **đầu vào (Input)** của mô hình.



Hình 2.2: Minh họa vị trí của Input trong sơ đồ

Ví dụ, với khách hàng B, đầu vào của mô hình là:

- x_1 = 29 (Tuổi)
- x_2 = 15 (Thu nhập)

- $x_3 = 1$ (Đã kết hôn)
- $x_4 = 0$ (Chưa có ô tô cũ)

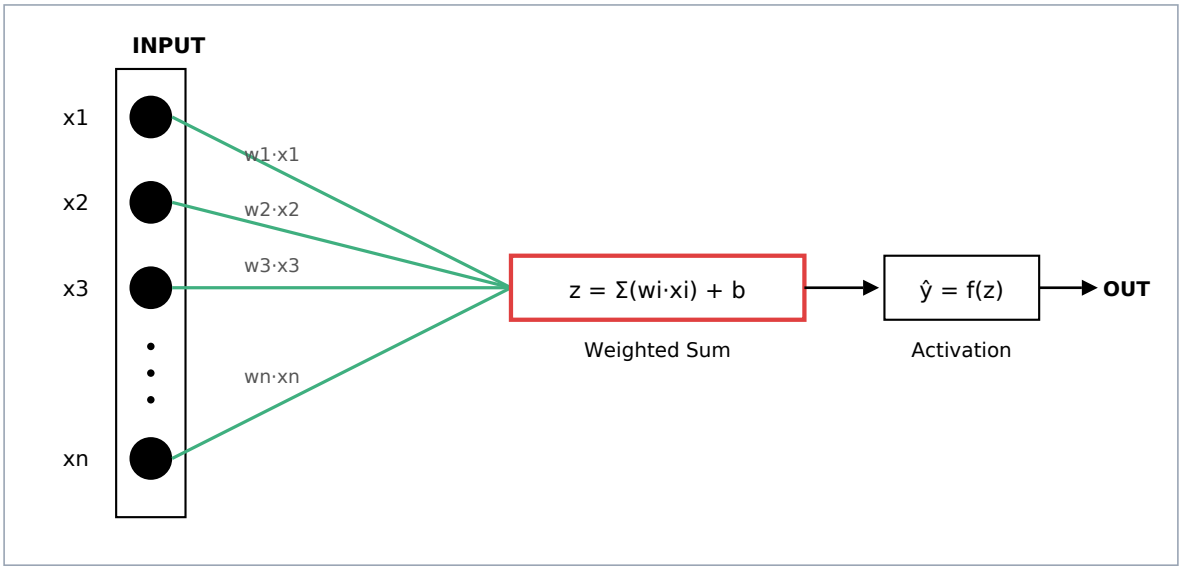
Hay có thể biểu diễn dưới dạng một vector:

$$x = [29, 15, 1, 0]$$

Nhiệm vụ của Perceptron là sử dụng các giá trị đầu vào này để dự đoán khách hàng có mua xe hay không. Lớp Input không thực hiện phép tính nào — nó chỉ đưa dữ liệu vào mô hình.

2.2.2 Weighted Sum (Tổ hợp tuyến tính)

Không phải tất cả các thông tin đều có mức độ ảnh hưởng như nhau đến quyết định mua xe. Chẳng hạn, **thu nhập** có thể quan trọng hơn **tuổi**, trong khi việc **đã kết hôn** cũng có thể tác động đến khả năng mua xe.



Hình 2.3: Minh họa vị trí của Weighted Sum trong sơ đồ

Có thể hiểu **trọng số (weight)** như mức độ ảnh hưởng của từng đặc trưng đến quyết định cuối cùng. Trong quá trình huấn luyện, Perceptron sẽ tự động điều chỉnh các trọng số này để đưa ra dự đoán ngày càng chính xác hơn.

Để thể hiện mức độ quan trọng của từng đặc trưng, Perceptron gán cho mỗi đầu vào một **trọng số (Weight)** tương ứng:

- Tuổi → w_1
- Thu nhập → w_2
- Đã kết hôn → w_3
- Có ô tô cũ → w_4

Ngoài ra, mô hình còn có một **bias (b)**, giúp điều chỉnh ngưỡng ra quyết định của Perceptron. Tất cả các giá trị này được kết hợp theo công thức:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

Trong đó:

- x_i là giá trị đầu vào
- w_i là trọng số tương ứng
- b là bias
- z là tổng có trọng số (Weighted Sum), hay còn gọi là tổ hợp tuyến tính

Ví dụ, giả sử tại một thời điểm trong quá trình huấn luyện, Perceptron có các tham số:

- $w_1 = 0.05$, $w_2 = 0.4$, $w_3 = 2$, $w_4 = -1$
- $b = -8$

Đối với khách hàng B, ta có:

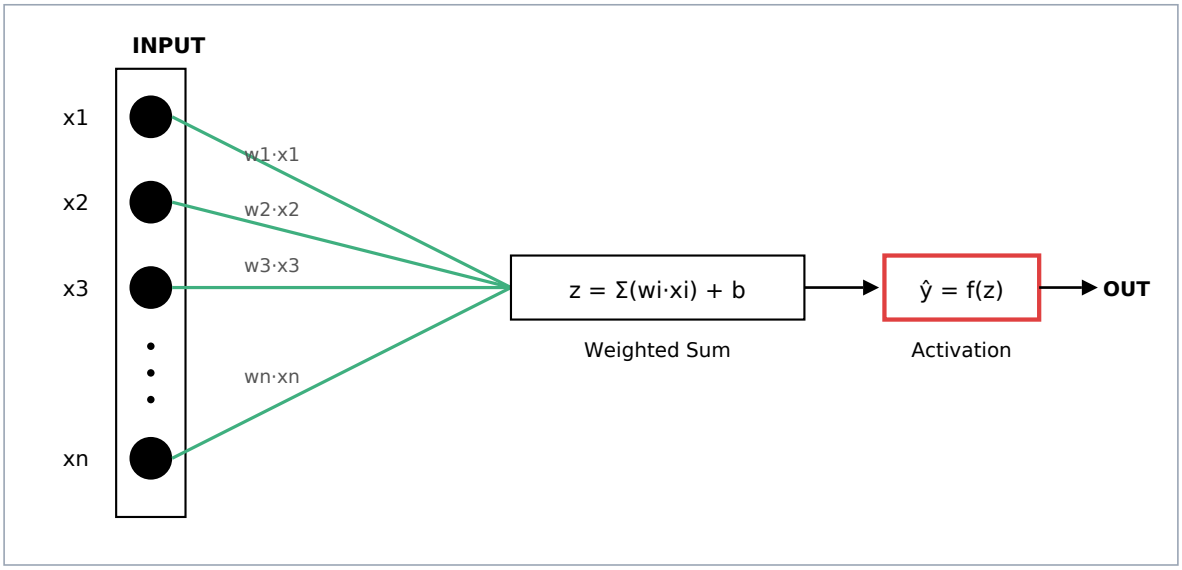
$$z = (0.05 \times 29) + (0.4 \times 15) + (2 \times 1) + (-1 \times 0) + (-8) = 1.45$$

Kết quả của bước này là một giá trị số thực. Tuy nhiên, giá trị này chưa cho biết khách hàng có mua xe hay không. Giá trị z sẽ tiếp tục được đưa qua hàm kích hoạt.

Lưu ý: Các trọng số và bias ở trên chỉ là ví dụ minh họa. Trong thực tế, chúng được mô hình tự động học từ dữ liệu trong quá trình huấn luyện.

2.2.3 Activation (Hàm kích hoạt)

Sau khi tính được giá trị z, Perceptron đưa giá trị này vào **hàm kích hoạt (Activation Function)** để đưa ra quyết định cuối cùng.



Hình 2.4: Minh họa vị trí của hàm kích hoạt trong sơ đồ

Đối với Perceptron, hàm kích hoạt thường là **hàm bước (Step Function)**:

Step Function:

- $\hat{y} = 1$ nếu $z \geq 0$
- $\hat{y} = 0$ nếu $z < 0$

Nói cách khác, trong ví dụ này:

- Nếu **z ≥ 0**, Perceptron dự đoán khách hàng **Có mua xe**.
- Nếu **z < 0**, Perceptron dự đoán khách hàng **Không mua xe**.

Như vậy, khách hàng B có z = 1.45. Do 1.45 ≥ 0, nên ŷ = 1. Điều này có nghĩa là Perceptron dự đoán **khách hàng B sẽ mua xe**.

2.2.4 Output (Đầu ra)

Đầu ra của Perceptron là giá trị dự đoán ŷ. Đối với bài toán phân loại nhị phân như ví dụ này:

- **ŷ = 1**: Dự đoán khách hàng **sẽ mua xe**.
- **ŷ = 0**: Dự đoán khách hàng **không mua xe**.

Sau khi đưa ra dự đoán, mô hình sẽ so sánh giá trị ŷ với **nhãn thực tế (y)** trong tập dữ liệu để đánh giá xem dự đoán đúng hay sai.

Nếu kết quả chưa chính xác, mô hình sẽ điều chỉnh các trọng số và bias để cải thiện dự đoán ở những lần học tiếp theo. Cơ chế cập nhật này sẽ được trình bày ở phần sau.

Lưu ý: Ký hiệu ŷ (đọc là "y-hat", hay "y mũ") được dùng để biểu diễn **giá trị dự đoán**, nhằm phân biệt với y là **nhãn thực tế** trong dữ liệu huấn luyện.

2.3 Tóm tắt quy trình lan truyền xuôi

Toàn bộ quá trình lan truyền xuôi của Perceptron có thể tóm tắt trong bốn bước:

Bước	Thành phần	Phép tính	Kết quả với khách hàng B
1	Input	Nhận vector đặc trưng x	x = [29, 15, 1, 0]
2	Weighted Sum	$z = \Sigma w_i x_i + b$	z = 1.45
3	Activation	ŷ = f(z), với f là hàm bước	f(1.45) = 1
4	Output	Trả về dự đoán	ŷ = 1 → Mua xe

Bảng 2.3: Tóm tắt các bước lan truyền xuôi

Viết gọn dưới dạng vector, toàn bộ Perceptron chỉ là một công thức:

$$\hat{y} = f(w \cdot x + b)$$

2.4 Perceptron giải quyết vấn đề gì?

Về mặt hình học, phương trình w · x + b = 0 biểu diễn một **đường thẳng** (trong không gian 2 chiều) hoặc một **siêu phẳng** (trong không gian nhiều chiều). Đường thẳng này chia không gian dữ liệu thành hai nửa: một bên là các điểm có z ≥ 0 (lớp 1), bên kia là các điểm có z < 0 (lớp 0).

Vì vậy, Perceptron giải quyết bài toán **phân loại nhị phân**: xếp mỗi mẫu dữ liệu vào một trong hai nhóm, như mua / không mua, cho vay / từ chối, thư rác / thư thường. Quá trình huấn luyện thực chất là **xoay và dịch chuyển đường phân chia** này cho đến khi nó tách đúng hai lớp dữ liệu.

Giới hạn quan trọng: *Perceptron chỉ giải được các bài toán mà dữ liệu* ***tách được tuyến tính*** *(linearly separable), tức là tồn tại một đường thẳng chia đúng hai lớp. Với bài toán như cổng XOR, không đường thẳng nào làm được điều đó — và đây chính là động lực để phát triển mạng nơ-ron nhiều lớp.*

3 Magic Hàm Kích Hoạt

Một điều thú vị là chỉ cần **thay đổi hàm kích hoạt**, cùng một cấu trúc "tổng có trọng số" có thể biến thành những mô hình học máy kinh điển khác. Điều này cho thấy hàm kích hoạt quyết định "tính cách" của mô hình.

3.1 Bỏ hàm kích hoạt, ta được Linear Regression

Nếu bỏ hàm kích hoạt đi (hay dùng hàm đồng nhất f(z) = z), đầu ra chính là tổng có trọng số:

$$\hat{y} = w \cdot x + b$$

Đây chính là mô hình **Hồi quy tuyến tính (Linear Regression)**. Đầu ra không còn là 0 hay 1, mà là một số thực bất kỳ, phù hợp cho bài toán **dự đoán giá trị liên tục** như giá nhà, doanh thu hay nhiệt độ. Trong ví dụ trên, thay vì dự đoán "có mua xe hay không", mô hình có thể dự đoán "khách hàng sẵn sàng chi bao nhiêu tiền cho chiếc xe".

3.2 Thay Step bằng Sigmoid, ta được Logistic Regression

Hàm bước có một nhược điểm: nó "nhảy" đột ngột từ 0 lên 1, không cho biết mô hình chắc chắn đến mức nào. Nếu thay hàm bước bằng **hàm Sigmoid**:

$$\sigma(z) = 1 / (1 + e^{-z})$$

thì đầu ra là một giá trị trơn nằm trong khoảng (0, 1), có thể hiểu là **xác suất** thuộc lớp 1. Mô hình khi đó chính là **Hồi quy Logistic (Logistic Regression)**. Với khách hàng B, $\sigma(1.45) \approx 0.81$, nghĩa là mô hình ước tính khoảng 81% khả năng khách hàng này mua xe.

Một lợi ích quan trọng khác: hàm Sigmoid **có đạo hàm ở mọi điểm**, trong khi hàm bước thì không. Nhờ đó, mô hình có thể được huấn luyện bằng phương pháp Gradient Descent — nền tảng của việc huấn luyện mạng nơ-ron nhiều lớp sau này.

3.3 Kết luận

Hàm kích hoạt	Đầu ra	Mô hình tương ứng	Bài toán phù hợp
Step (hàm bước)	0 hoặc 1	Perceptron	Phân loại nhị phân
Không có (f(z) = z)	Số thực bất kỳ	Linear Regression	Dự đoán giá trị liên tục
Sigmoid	Xác suất trong (0, 1)	Logistic Regression	Phân loại kèm xác suất

Bảng 3.1: Cùng một cấu trúc, khác hàm kích hoạt

Ba mô hình này có chung "bộ khung" là tổng có trọng số $w \cdot x + b$. Điểm khác biệt duy nhất nằm ở hàm kích hoạt — đó chính là "phép màu" của hàm kích hoạt.

4 Perceptron Học Như Thế Nào?

4.1 Ý tưởng cốt lõi

Perceptron học theo nguyên tắc rất tự nhiên: **học từ sai lầm**. Mô hình lần lượt xem từng mẫu dữ liệu, đưa ra dự đoán, rồi so sánh với nhãn thực tế:

- Nếu **dự đoán đúng**: giữ nguyên trọng số.
- Nếu **dự đoán sai**: điều chỉnh trọng số theo hướng giúp lần sau dự đoán đúng hơn.

Giống như một học sinh làm bài tập: làm đúng thì giữ cách làm cũ, làm sai thì xem lại lỗi và sửa.

4.2 Công thức cập nhật trọng số

$$w_i \leftarrow w_i + \eta\,(y - \hat{y})\,x_i$$
$$b \leftarrow b + \eta\,(y - \hat{y})$$

Trong đó:

- **η** (eta) là **tốc độ học (learning rate)**, thường là một số dương nhỏ như 0.1 hay 0.01, quyết định mỗi lần sửa nhiều hay ít.
- **$(y - \hat{y})$** là **sai số** của dự đoán.

Sai số $(y - \hat{y})$ chỉ có thể nhận ba giá trị, và mỗi trường hợp có ý nghĩa rõ ràng:

y	$\hat{y}$	$y - \hat{y}$	Ý nghĩa	Trọng số thay đổi
1	1	0	Dự đoán đúng	Giữ nguyên
0	0	0	Dự đoán đúng	Giữ nguyên
1	0	+1	Dự đoán thấp (bỏ sót)	Tăng theo hướng x
0	1	−1	Dự đoán cao (báo nhầm)	Giảm theo hướng x

Bảng 4.1: Các trường hợp của sai số

Ví dụ: Với các tham số ở mục 2.2.2, xét khách hàng A có $x = [24, 8, 0, 0]$ và $y = 0$. Ta có $z = 0.05 \times 24 + 0.4 \times 8 + 0 + 0 - 8 = -3.6$, nên $\hat{y} = 0$. Dự đoán đúng, trọng số giữ nguyên. Giả sử ở một mẫu khác mô hình dự đoán $\hat{y} = 1$ trong khi $y = 0$, thì với $\eta = 0.01$, mỗi trọng số sẽ bị trừ đi $0.01 \times x_i$ và bias giảm 0.01, khiến z của mẫu đó nhỏ đi, đẩy dự đoán về phía 0.

4.3 Các bước của thuật toán

- **Bước 1: Khởi tạo:** Gán giá trị ban đầu cho trọng số w và bias b (thường bằng 0 hoặc số ngẫu nhiên nhỏ), chọn tốc độ học η.
- **Bước 2: Lan truyền xuôi:** Với mỗi mẫu (x, y) trong dữ liệu, tính $z = w \cdot x + b$ và $\hat{y} = f(z)$.
- **Bước 3: Tính sai số:** So sánh dự đoán với nhãn thực tế: sai số $= y - \hat{y}$.
- **Bước 4: Cập nhật:** Điều chỉnh w và b theo công thức ở mục 4.2.

- **Bước 5: Lặp lại:** Duyệt hết dữ liệu được gọi là một epoch. Lặp lại nhiều epoch cho đến khi mô hình không còn dự đoán sai, hoặc đạt số epoch tối đa.

> ***Định lý hội tụ Perceptron:*** *Nếu dữ liệu tách được tuyến tính, thuật toán chắc chắn tìm ra một đường phân chia đúng sau một số hữu hạn bước cập nhật.*

5 Tổng Kết Bài Học

Qua bài học này, chúng ta đã đi từ câu hỏi "vì sao cần Perceptron" đến cách nó hoạt động và cách nó học:

- **Perceptron** là mô hình học máy có giám sát đơn giản nhất và là đơn vị cơ bản của mạng nơ-ron nhân tạo. Nó ra đời từ ý tưởng **học quy luật từ dữ liệu** thay vì lập trình sẵn mọi quy tắc.
- Perceptron được gọi là **Artificial Neuron** vì mô phỏng cách tế bào thần kinh tổng hợp tín hiệu và "phát xung" khi vượt ngưỡng.
- Cấu tạo gồm bốn thành phần: **Input → Weighted Sum → Activation → Output**, xử lý theo một chiều gọi là **lan truyền xuôi**: $\hat{y} = f(w \cdot x + b)$.
- Hàm kích hoạt quyết định bản chất mô hình: bỏ đi ta được **Linear Regression**, thay bằng Sigmoid ta được **Logistic Regression**.
- Perceptron **học từ sai lầm** theo quy tắc $w \leftarrow w + \eta(y - \hat{y})x$, và chắc chắn hội tụ khi dữ liệu tách được tuyến tính.
- Giới hạn lớn nhất là chỉ giải được bài toán **tách tuyến tính**. Để vượt qua giới hạn này, ta ghép nhiều Perceptron thành **mạng nơ-ron nhiều lớp (MLP)** — chủ đề của các bài học tiếp theo.